

Highlight Cutter

System Design & Deployment Report

Meeting crosstalk removal & YouTube highlight-reel generator
Architecture · Component reference · Dependency inventory · Deployment requirements

September 18, 2026
Prepared for: FloLabs Automation

Contents

Contents	2
1. Executive Summary	4
Verification status	4
What this report covers	4
2. System Architecture	5
3. Component Reference	7
3.1 core/ — shared config, models, logging	7
3.2 ingest/ — getting a video onto local disk	7
3.3 transcribe/ — ASR, diarization, overlap flags	8
3.4 decide/ — the one LLM call in the pipeline	8
3.5 edl/ — turning decisions into a safe cut list	9
3.6 slice/ — ffmpeg extraction, concat, transcript remap	9
3.7 pipeline.py — orchestration	10
3.8 api/ — HTTP surface	10
3.9 web/index.html — UI	10
4. Job Lifecycle & Data Layout	12
Status states (core/models.py: JobStatus)	12
On-disk layout	12
5. Third-Party Dependency Inventory	13
5.1 External services — network calls at request time	13
5.2 Self-hosted open-source libraries — run on our own compute	13
5.3 Notable transitive dependencies	14
6. What Runs on Our Own System vs. a Third Party	15
7. Deployment Requirements	16
7.1 Runtime environment	16

7.2 Required configuration (environment variables)	16
7.3 Current limitations (as built — fine for a demo, not for production)	17
7.4 Recommended path to a real deployment	17
8. Issues Hit & Resolved While Building This	18
Appendix A — Full Pinned Dependency List	19

1. Executive Summary

Highlight Cutter is a working, end-to-end pipeline that takes a raw meeting recording or YouTube video and produces a trimmed output video with a real web UI on top. It runs in two selectable modes:

Highlights mode Aggressive cut — keeps only genuinely highlight-worthy moments (a strong point, a punchline, a key result). Most segments are removed by design.	Crosstalk / Light cleanup mode Conservative cut — keeps everything except crosstalk, interruptions, and filler; removes only low-value overlapping speech.
--	--

Word-level ASR timestamps decide what to cut; ffmpeg does the actual cutting; no audio-level ML noise detection is used. Overlap detection is a cheap deterministic heuristic — the only LLM call in the whole pipeline is the single keep/remove judgment pass, which keeps per-video cost and latency low.

Verification status

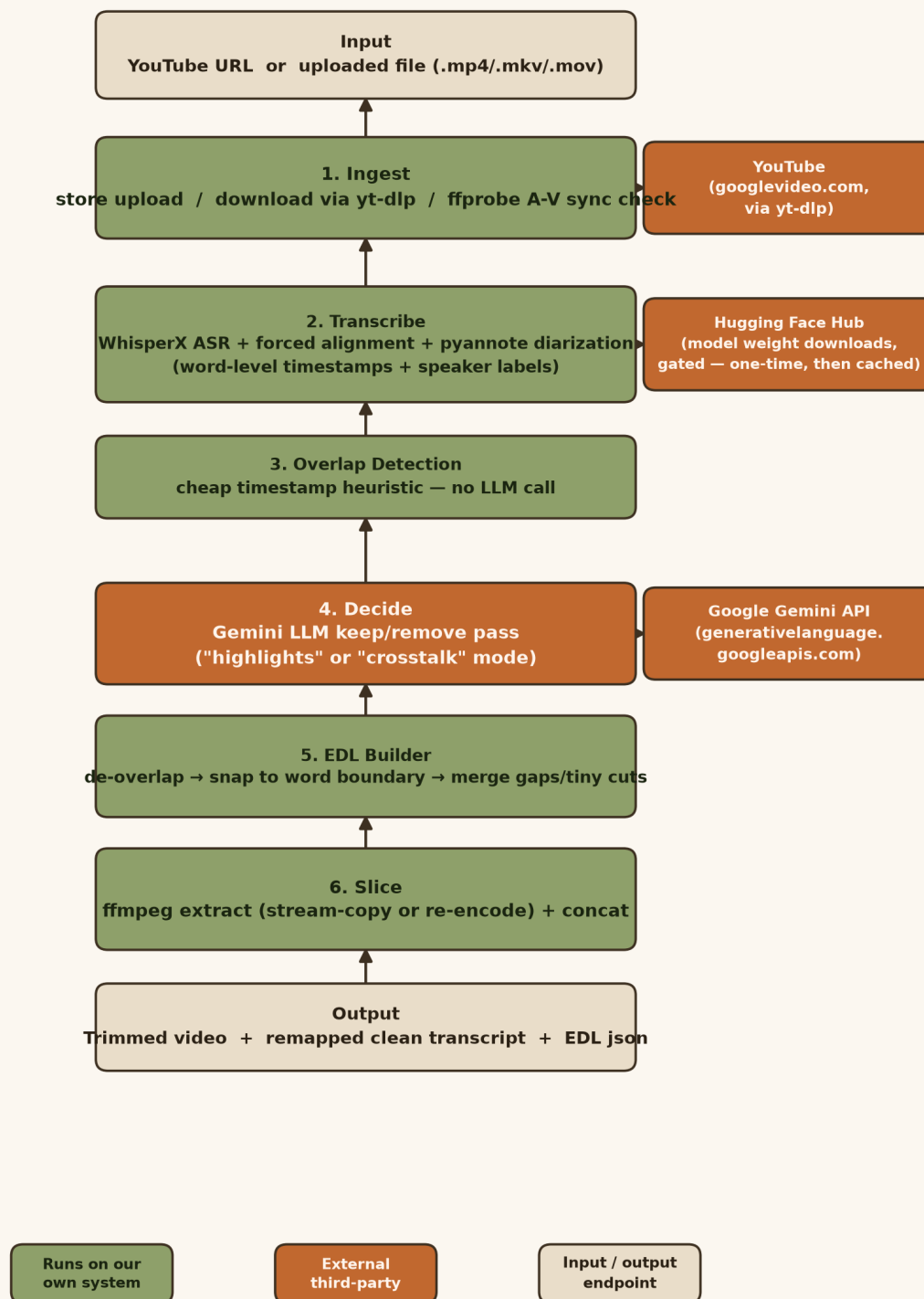
- 37 automated tests pass, covering all deterministic logic (overlap detection, EDL sort/de-overlap/snap/merge, transcript remapping, API request/response wiring) with WhisperX, yt-dlp and Gemini calls mocked out.
- The full pipeline has been run end-to-end for real against a live YouTube video: real download, real WhisperX transcription + diarization, a real Gemini API call, real ffmpeg slicing — output duration matched the Edit Decision List's kept ranges.
- The web UI (YouTube URL or drag-and-drop file upload, live progress, video preview, download) has been exercised against the running API.

What this report covers

- System architecture and the full processing pipeline (Section 2).
- What every component does and how (Section 3).
- Job lifecycle and on-disk data layout (Section 4).
- Full third-party dependency inventory — external services vs. self-hosted libraries (Section 5).
- A direct answer to “what runs on our own system vs. a third party” (Section 6).
- What is needed to deploy this for real, and its current limitations (Section 7).
- Issues actually hit while building and verifying this, and how they were resolved (Section 8).

2. System Architecture

The system is a single FastAPI service with six sequential processing stages, a thin job-tracking layer, and a static web UI served from the same process. Each stage is an independent Python package with one job; **pipeline.py** is the only module that wires them together in order.



Highlight Cutter — processing pipeline & system boundary

Figure 1. Processing pipeline. Green = runs on our own system (local compute, no outbound call other than a one-time model download). Terracotta = depends on an external third-party service at request time.

3. Component Reference

One subsection per module. “Key files” are repo-relative paths; “Depends on” lists only what that specific module imports/calls, not the whole app.

3.1 core/ — shared config, models, logging

What it does

Central place for environment-driven settings (pydantic-settings), the pydantic data models shared by every stage (Word, Segment, Decision, EDLRange, EditDecisionList, JobRecord, JobStatus), and a thin logging setup.

How it works

Settings() reads .env once via SettingsConfigDict(env_file=".env"); get_settings() is @lru_cache'd so it's read once per process. On first call it also creates storage/videos and storage/jobs, and — notably — injects the directory containing FFMPEG_BIN/FFPROBE_BIN onto this process's PATH, so libraries like WhisperX that shell out to a bare “ffmpeg” still find it even if the OS-level PATH wasn't refreshed after a fresh ffmpeg install.

Key files

core/config.py, core/models.py, core/logging.py

Depends on

pydantic, pydantic-settings

3.2 ingest/ — getting a video onto local disk

What it does

Three independent jobs: persist an uploaded file, download a YouTube URL, and validate a source file before it enters the pipeline.

How it works

store_video() writes an uploaded file to storage/videos/{uuid}.ext after checking the extension is one of .mp4/.mkv/.mov. download_youtube() drives yt-dlp to fetch best video+audio and mux to mp4, resolving ffmpeg's location explicitly rather than trusting PATH. validate_video() shells out to ffprobe, requires both a video and an audio stream to exist, and raises AVSyncError if their reported durations drift by more than 1.0s — the pipeline treats that as a hard stop (status=skipped_desync) rather than silently producing a bad cut.

Key files

ingest/store.py, ingest/youtube.py, ingest/validate.py

Depends on

yt-dlp (download); ffprobe binary (validation, via subprocess)

3.3 transcribe/ — ASR, diarization, overlap flags

What it does

Turns an audio/video file into word-level {word, start, end, speaker} tuples, then flags which adjacent words from different speakers overlap in time.

How it works

transcribe() runs WhisperX's three-stage flow: (1) faster-whisper ASR for word timings, (2) forced alignment with a wav2vec2 model for frame-accurate word boundaries, (3) pyannote speaker diarization, then merges diarization output onto each word via whisperx.assign_word_speakers(). The whisperx import is deliberately lazy (inside the function) so the rest of the service can run and be tested without torch installed. flag_overlaps() is a separate, cheap, deterministic pass: words are sorted by start time and adjacent words from different speakers are flagged overlap_candidate=True when start_b < end_a — plain arithmetic, no model call, per the design goal of not spending LLM calls detecting that overlap happened.

Key files

transcribe/whisperx_backend.py, transcribe/overlap.py

Depends on

WhisperX (orchestration), which pulls in torch, faster-whisper, pyannote-audio; downloads model weights from Hugging Face on first use

3.4 decide/ — the one LLM call in the pipeline

What it does

Groups words into speaker-turn segments, sends them to Gemini, and gets back a keep/remove decision with a reason for every segment.

How it works

build_segments() collapses consecutive same-speaker words into a Segment (speaker, start, end, text, overlap_candidate). get_decisions() picks one of two system prompts by mode ("highlights" = aggressive, keep only clip-worthy moments; "crosstalk" = conservative, keep everything except low-value overlap) and calls the Gemini API with response_mime_type="application/json", validating the response against a pydantic schema (list[Decision]). On a malformed response, a decision-count mismatch, or a transient Gemini 5xx/429 (google.genai.errors.APIError), it retries once with a stricter reminder prompt; a second failure raises DecisionError and fails the job loudly rather than guessing.

Key files

decide/segments.py, decide/gemini_client.py

Depends on

google-genai SDK → Google Gemini API (network call, billed)

3.5 edl/ — turning decisions into a safe cut list

What it does

Converts the LLM's raw, untrusted decision list into a clean, sorted, non-overlapping Edit Decision List (EDL) of KEEP ranges ready for ffmpeg.

How it works

A fixed pipeline of pure functions: (1) keep only decision=="keep" ranges and sort by start — the LLM's ordering is never trusted; (2) coalesce any overlapping/touching ranges; (3) snap every start/end to the nearest word boundary from the transcript, expanding outward (never inward) so a cut never clips a word in half; (4) coalesce again, since snapping can push neighbors into overlap; (5) merge KEEP ranges separated by less than a configurable gap (default 0.3s) to avoid choppy micro-cuts; (6) merge or drop KEEP ranges under a configurable minimum duration (default 0.5s). If every range ends up filtered out (e.g. the LLM removed everything), it falls back to one KEEP range spanning the whole source — a safe no-op rather than an empty output video. `validate_edl()` asserts the final result is sorted and non-overlapping.

Key files

`edl/builder.py`

Depends on

Nothing external — pure Python logic over pydantic models

3.6 slice/ — ffmpeg extraction, concat, transcript remap

What it does

Physically cuts the source video down to the EDL's KEEP ranges, stitches the pieces back together, and remaps the original transcript onto the new, shorter timeline.

How it works

`probe_keyframe_timestamps()` runs a cheap keyframe-only `ffprobe` pass once per source file. For each KEEP range, `choose_mode()` picks stream-copy ("-c copy", fast) when a keyframe is within 0.5s of the cut point, or re-encode (libx264/aac, slower but frame-accurate) otherwise. Clips are concatenated with ffmpeg's concat demuxer when codecs match, falling back to the concat filter (re-encoding everything) if that fails — exactly the two strategies the original design called for. `remap_transcript()` walks the KEEP ranges in order, assigns each original word to whichever range fully contains it, and shifts its timestamps by that range's cumulative offset in the output; words that were cut are simply dropped. All intermediate per-segment clip files are deleted in a finally block, even on failure.

Key files

`slice/ffmpeg_wrapper.py`, `slice/pipeline.py`, `slice/transcript.py`

Depends on

ffmpeg / ffprobe binaries (via subprocess) — not a Python package

3.7 pipeline.py — orchestration

What it does

The only module that calls all six stages in order for a given job, updating job status after each stage so the API/UI can show live progress.

How it works

`run_pipeline()` validates A/V sync first (short-circuits to `status=skipped_desync` on failure), then walks transcribing → deciding → building_edl → slicing → done, writing `transcript.json` / `edl.json` / `clean_transcript.json` to the job's folder as it goes. Any exception anywhere in that chain is caught, logged, and turns into `status=failed` with the error message attached to the job record — nothing propagates up and crashes the background task silently. `run_youtube_pipeline()` is a thin wrapper that downloads first (`status=downloading`) and then calls `run_pipeline()` against the local file.

Key files

`pipeline.py`

Depends on

Every stage module above

3.8 api/ — HTTP surface

What it does

FastAPI app exposing the pipeline over HTTP for the web UI and for external orchestrators (e.g. n8n) to call.

How it works

`POST /jobs` (multipart upload) and `POST /jobs/youtube` (JSON body) both create a `JobRecord`, persist it via `JobStore`, and hand the actual work to FastAPI's `BackgroundTasks` so the HTTP response returns immediately with a `job_id`. `JobStore` is a small thread-safe wrapper: an in-memory dict for fast reads plus a JSON file per job (`storage/jobs/{id}/job.json`) so a job's last-known state survives a lookup after a process restart (though a job that was actively running at restart time does not resume — see Section 7). `GET /jobs/{id}` returns current status; `GET /jobs/{id}/video`, `/edl`, and `/transcript` serve the outputs once `status=done`. `GET /` serves the static web UI file directly.

Key files

`api/main.py`, `api/jobs.py`

Depends on

FastAPI, Starlette, uvicorn, python-multipart

3.9 web/index.html — UI

What it does

A single-file vanilla HTML/CSS/JS page: paste a YouTube URL or drag-and-drop a file, pick a mode, watch live progress, preview and download the result.

How it works

No build step, no framework — plain `fetch()` calls against the API above. Polls `GET /jobs/{id}` every 3s and renders a step list that adapts (the “Downloading video” step is hidden entirely for direct file uploads, since that stage never runs for them). On `status=done` it points a `<video>` element and download link straight at `/jobs/{id}/video`.

Key files

`web/index.html`

Depends on

Nothing — no external JS libraries, fonts, or CDNs

4. Job Lifecycle & Data Layout

Status states (core/models.py: JobStatus)

Status	Meaning
queued	Job record created, background task not yet started.
downloading	YouTube-only: yt-dlp is fetching the source video.
transcribing	WhisperX ASR + alignment + diarization running.
deciding	Gemini keep/remove pass in progress.
building_edl	Sorting/snapping/merging into the final EDL.
slicing	ffmpeg extracting and concatenating the output video.
done	output_video_path, edl_path, transcript_path, clean_transcript_path all set.
failed	error holds the exception message; any stage can land here.
skipped_desync	A/V duration mismatch > 1.0s — pipeline refuses to process it.

On-disk layout

storage/	
videos/{video_id}.mp4	uploaded or downloaded source files
jobs/{job_id}/	
job.json	persisted JobRecord (status, paths, error)
transcript.json	raw word-level transcript (pre-cut)
edl.json	final EditDecisionList
output.mp4	the trimmed result
clean_transcript.json	transcript remapped onto the trimmed timeline
tmp/	per-segment ffmpeg clips (deleted after slicing)

5. Third-Party Dependency Inventory

Two different kinds of “third-party” are mixed together in most stacks; this report keeps them separate because they have very different operational and cost implications.

5.1 External services — network calls at request time

These require an account/API key, cost money or are rate-limited, and the pipeline cannot function without reaching them over the network.

Service	Used by	What for	Auth / cost
Google Gemini API	decide/gemini_client.py	The single LLM call per job: judges keep/remove for every speaker-turn segment.	GEMINI_API_KEY, billed per token. Model pinned to gemini-3.5-flash-lite — see Section 8 for why.
Hugging Face Hub	transcribe/whisperx_backend.py	Downloads WhisperX/pyannote model weights (ASR, alignment, diarization). Cached to disk after the first run — not a per-job network call.	HF_TOKEN (read scope) from an account that has accepted the gated terms on 3 model repos. Downloads themselves are free.
YouTube (googlevideo.com)	ingest/youtube.py	Source video download when a YouTube URL is submitted instead of a file upload.	No auth; unofficial (yt-dlp scrapes YouTube's player — no SLA, can break on YouTube-side changes).

5.2 Self-hosted open-source libraries — run on our own compute

Installed locally, execute locally, no external call at runtime (aside from the one-time model download above).

Package	Role
whisperx (git, m-bain/whisperX)	ASR + alignment + diarization orchestration — the core transcription engine.
torch / torchaudio / torchvision	ML runtime WhisperX and pyannote run on (CPU here; GPU-capable).
faster-whisper + ctranslate2	The actual Whisper speech-to-text inference engine (CTranslate2-optimized).
pyannote-audio (+ pyannote-core/-pipeline/-database/-metrics)	Speaker diarization models and pipeline framework.
ffmpeg / ffprobe	Video/audio decoding, cutting, re-encoding, concatenation, and stream probing. External binary, not pip-installed.
fastapi + starlette + uvicorn	HTTP API and ASGI server.
pydantic + pydantic-settings	Request/response/data validation and typed environment config.
python-multipart	Multipart form parsing for the file-upload endpoint.
yt-dlp	YouTube download engine (runs locally; talks to YouTube's servers).

Package	Role
google-genai	Official SDK used to call the Gemini API (the SDK is local; the model it calls is not).
ruff, pytest, httpx	Dev-only: linting and the test suite (not shipped/run in production).

5.3 Notable transitive dependencies

The direct packages above pull in roughly 90 further packages automatically (full pinned list in Appendix A). The ones worth knowing about by name:

Package	Why it's there
numpy, scipy, scikit-learn	Numerical backbone for torch/pyannote.
transformers, tokenizers, huggingface_hub, hf-xet	Model loading/download machinery used by WhisperX/pyannote.
onnxruntime	Runs some of pyannote's models in ONNX form.
pytorch-lightning, torchmetrics, optuna	Training/eval framework dependencies pulled in by pyannote-audio (not used at inference time by this app, but still installed).
nltk	Text tokenization used internally by WhisperX/pyannote.
pandas, matplotlib	Used internally by pyannote for data handling/plotting, not by our own code.

6. What Runs on Our Own System vs. a Third Party

Direct answer, stage by stage:

Pipeline stage	Runs on our system	Calls a third party
Ingest — file upload	Yes — disk write only	No
Ingest — YouTube URL	Download orchestration runs locally (yt-dlp)	Yes — fetches the actual video bytes from YouTube/googlevideo.com
A/V sync validation	Yes — local ffprobe	No
Transcription (ASR)	Yes — model runs on our CPU/GPU	Weights fetched from Hugging Face once, then cached locally
Diarization	Yes — model runs on our CPU/GPU	Weights fetched from Hugging Face once (3 separate gated repos), then cached locally
Overlap detection	Yes — plain arithmetic	No
Decide (keep/remove)	Prompt building only	Yes — every job makes a live call to the Gemini API
EDL building	Yes — pure Python	No
Slicing / concatenation	Yes — local ffmpeg	No
Job storage / API / UI	Yes — local disk + in-process server	No

Bottom line: the only per-job dependency on an external service that the pipeline cannot work around is the Gemini decide call. YouTube download is external but only exercised when a URL (not a file) is submitted. Model weight downloads from Hugging Face happen once and are then fully local. Everything else — ASR compute, diarization compute, video decoding/cutting, orchestration, storage, and the UI — runs entirely on whatever machine this is deployed to.

7. Deployment Requirements

7.1 Runtime environment

Requirement	Detail
Python version	3.10–3.12 (hard constraint — the whisperx package declares requires-python <3.14,>=3.10; this was discovered directly when installation silently failed on 3.14 during setup).
OS binaries	ffmpeg and ffprobe must be reachable — either on PATH, or point FFMPEG_BIN/FFPROBE_BIN at absolute paths.
CPU	Works on CPU alone with the “small” Whisper model + int8 quantization (WHISPERX_COMPUTE_TYPE=int8). This is what was verified.
GPU (optional)	WHISPERX_DEVICE=cuda + a larger model (e.g. large-v3) for materially faster transcription at higher accuracy; recommended for production throughput, not required to run.
RAM	Comfortable CPU inference for WhisperX + pyannote wants 8GB+ available to the process; more for larger Whisper models or concurrent jobs.
Disk — model cache	One-time download, then cached under the Hugging Face cache dir: faster-whisper “small” (~250–500MB), the wav2vec2 alignment model (~360MB), and the diarization pipeline’s several component models (segmentation, embedding, PLDA) — roughly 1–2GB combined, observed directly during setup.
Disk — working storage	Grows with usage: every uploaded/downloaded source video plus every job’s transcript/EDL/output video are kept indefinitely under storage/ (see Section 7.3).

7.2 Required configuration (environment variables)

Variable	Purpose	Notes
HF_TOKEN	Hugging Face read token	Account must accept gated terms on all 3 diarization-chain repos (Section 8).
GEMINI_API_KEY	Gemini API auth	Billed per token.
GEMINI_MODEL	Which Gemini model to call	Default gemini-3.5-flash-lite (see Section 8 for why not a newer “flash” model).
WHISPERX_MODEL	Whisper model size	Default “small” — CPU-friendly.
WHISPERX_DEVICE	cpu or cuda	Default cpu.
WHISPERX_COMPUTE_TYPE	Quantization	Default int8 for fast CPU inference.
WHISPERX_DIARIZE_MODEL	Diarization pipeline name	Default pyannote/speaker-diarization-3.1.
STORAGE_DIR	Where videos/ and jobs/ live	Default ./storage.
FFMPEG_BIN / FFPROBE_BIN	Binary locations	Default bare “ffmpeg”/“ffprobe” (PATH lookup); set to absolute paths if PATH is unreliable.

7.3 Current limitations (as built — fine for a demo, not for production)

- **In-process job queue.** Jobs run via FastAPI BackgroundTasks inside the same process that serves HTTP requests. There is no worker pool and no cross-process queue — this does not scale horizontally, and a job actively running when the process restarts is abandoned mid-status (observed directly: a job was left stuck at status=transcribing after a manual server restart during testing).
- **Job state is JSON-on-disk, not a database.** Fine for a single-box demo; a concurrent-write race or partial write under real load has not been hardened against.
- **No authentication or rate-limiting** on any API endpoint. Anyone who can reach the port can submit jobs, burn Gemini API budget, and read any job's output.
- **No storage retention policy.** storage/videos and storage/jobs grow forever; nothing prunes old jobs or source files.
- **Single Gemini call per job, no batching.** Fine at demo volume; at scale this is the primary per-job cost and the primary external latency/availability dependency (Section 8 covers a real outage hit during testing).

7.4 Recommended path to a real deployment

- Replace BackgroundTasks with a real task queue (Celery, RQ, or Arq) backed by Redis so jobs survive a process restart and can run on multiple workers.
- Move job state from JSON files to a real database (Postgres is the obvious choice given pydantic models translate directly).
- Move video storage to an object store (S3-compatible) once this runs on more than one box, or once local disk growth becomes a problem.
- Put the API behind authentication before exposing it beyond localhost.
- Add a retention job that deletes old storage/videos and storage/jobs after N days.
- Containerize: a Docker image needs Python 3.12, ffmpeg installed at the OS level, the torch/whisperx dependency chain, and a persistent volume mounted at the Hugging Face cache directory so model weights aren't re-downloaded on every container restart.
- For real throughput, deploy on a GPU-backed instance and set WHISPERX_DEVICE=cuda with a larger Whisper model — CPU inference is workable for a demo, not for concurrent production load.
- Add monitoring/alerting on the Gemini call specifically — it is both the single point of external failure and the only per-job cost.

8. Issues Hit & Resolved While Building This

Included because each of these is a real, currently-true fact about the third-party dependencies above, discovered by actually running the system rather than assumed from documentation.

8.1 WhisperX's Python version ceiling

whisperx declares `requires-python <3.14,>=3.10`; installation into a Python 3.14 environment failed outright. Resolved by installing Python 3.12 specifically for this project's virtual environment.

8.2 Diarization needs three accepted gated models, not two

pyannote/speaker-diarization-3.1's own pipeline config internally pulls PLDA weights from a newer gated repo, pyannote/speaker-diarization-community-1. Accepting terms on speaker-diarization-3.1 and segmentation-3.0 alone was not sufficient — the diarization call failed with a 403 until terms were also accepted on the community-1 repo specifically.

8.3 WhisperX API surface changed under a moving git dependency

requirements.txt pins whisperx to the m-bain/whisperX git HEAD (there is no tagged stable release with the required Python 3.12 compatibility at time of writing). The installed build (3.8.7rc1) renamed DiarizationPipeline's `use_auth_token` parameter to `token`, and changed its default diarization model. This is a real risk of pinning to a git branch: the API can and did change under an unpinned commit.

8.4 Gemini model availability and deprecation churn

Three different Gemini model identifiers were tried in sequence during verification: `gemini-2.5-flash` returned 404 ("no longer available to new users"); `gemini-3.6-flash` and `gemini-flash-latest` both returned sustained 503 UNAVAILABLE ("high demand") — plausibly because Google is funneling newly-restricted accounts onto very recently released models. `gemini-3.5-flash-lite` responded reliably and is what the app now defaults to. The decide client was also hardened to retry once on a transient Gemini 5xx/429, not just on malformed JSON output.

8.5 Host PATH not visible to already-running/freshly-spawned processes

Installing `ffmpeg` via `winget` updates the persistent user `PATH`, but a process already running (or a new child process spawned by a long-running parent that hasn't reloaded its own environment) does not automatically see it. Two mitigations were applied: `FFMPEG_BIN/FFPROBE_BIN` can be set to absolute paths, and `core/config.py` proactively injects that directory onto the current process's `PATH` so that libraries which shell out to a bare `"ffmpeg"` (WhisperX's internal audio loader; `yt-dlp`'s muxing step) still find it.

Appendix A — Full Pinned Dependency List

Complete output of `pip list --format=freeze` against the working virtual environment, for exact reproducibility.

Package	Version	Package	Version
Jinja2	3.1.6	Mako	1.4.1
MarkupSafe	3.0.3	PyYAML	6.0.3
Pygments	2.21.0	SQLAlchemy	2.0.54
aiohappyeyeballs	2.7.1	aiohttp	3.14.3
aiosignal	1.4.0	alembic	1.20.0
annotated-doc	0.0.5	annotated-types	0.8.0
antlr4-python3-runtime	4.9.3	anyio	4.15.1
asteroid-filterbanks	0.4.0	attrs	26.1.0
av	18.1.0	certifi	2026.7.22
cffi	2.1.1	charset-normalizer	3.5.1
click	8.5.0	cloudpickle	3.1.2
colorama	0.4.6	colorlog	6.12.0
contourpy	1.4.0	cryptography	50.0.1
ctranslate2	4.8.2	cycler	0.12.1
defusedxml	0.7.1	distro	1.9.0
einops	0.8.2	fastapi	0.141.1
faster-whisper	1.2.1	filelock	4.0.0
flatbuffers	25.12.19	fonttools	4.65.0
frozenset	1.8.0	fsspec	2026.7.0
google-auth	2.58.0	google-genai	2.24.0
googleapis-common-protos	1.75.3	greenlet	3.5.6
grpcio	1.84.0	h11	0.16.0
hf-xet	1.6.0	httpcore	1.0.9
httptools	0.8.0	httpx	0.28.1
huggingface_hub	1.32.0	idna	3.20
iniconfig	2.3.0	joblib	1.6.0
julius	0.2.8	kiwisolver	1.5.1
lightning-utilities	0.15.3	lightning	2.6.6
markdown-it-py	4.2.0	matplotlib	3.11.2
mdurl	0.1.2	mpmath	1.3.0
multidict	6.8.0	narwhals	2.26.0
networkx	3.6.1	nlTK	3.10.3
numpy	2.5.3	omegaconf	2.3.1
onnxruntime	1.30.0	opentelemetry-api	1.44.0

Package	Version	Package	Version
opentelemetry-exporter-otlp-proto-common	1.44.0	opentelemetry-exporter-otlp-proto-grpc	1.44.0
opentelemetry-exporter-otlp-proto-http	1.44.0	opentelemetry-exporter-otlp	1.44.0
opentelemetry-proto	1.44.0	opentelemetry-sdk	1.44.0
opentelemetry-semantic-conventions	0.65b0	optuna	5.0.0
packaging	26.3	pandas	3.0.5
pillow	12.3.0	pluggy	1.6.0
primePy	1.3	propcache	0.5.4
protobuf	7.36.1	pyannote-audio	4.0.7
pyannote-core	6.0.1	pyannote-database	6.1.1
pyannote-metrics	4.1	pyannote-pipeline	4.0.0
pyannoteai-sdk	0.4.0	pyasn1	0.6.4
pyasn1_modules	0.4.2	pycparser	3.0
pydantic-settings	2.15.0	pydantic	2.13.5
pydantic_core	2.46.5	pyparsing	3.3.2
pytest	9.1.1	python-dateutil	2.9.0.post0
python-dotenv	1.2.3	python-multipart	0.0.32
pytorch-lightning	2.6.6	pytorch-metric-learning	2.9.0
regex	2026.9.10	requests	2.34.2
rich	15.0.0	ruff	0.16.8
safetensors	0.8.0	scikit-learn	1.9.1
scipy	1.18.1	setuptools	84.0.0
shellingham	1.5.4	six	1.17.0
sniffio	1.3.1	sortedcontainers	2.4.0
starlette	1.6.0	sympy	1.14.0
tenacity	9.1.4	threadpoolctl	3.7.0
tokenizers	0.23.2	torch-audiomentations	0.12.0
torch	2.8.0	torch_pitch_shift	1.2.5
torchaudio	2.8.0	torchcodec	0.7.0
torchmetrics	1.9.0	torchvision	0.23.0
tqdm	4.70.1	transformers	5.17.0
typer	0.27.2	typing-inspection	0.4.4
typing_extensions	4.16.0	tzdata	2026.4
urllib3	2.8.0	uvicorn	0.53.0
watchfiles	1.2.0	websockets	16.1.1
whisperx	3.8.7rc1	yaml	1.25.1
yt-dlp	2026.8.19		

Total: 139 packages (8 declared directly in requirements.txt; the remainder are transitive, mostly pulled in by the WhisperX/torch/pyannote dependency chain).